Figure 10.5 The prompt chaining flow

Listing 10.12 shows the YAML pseudocode that describes the flow in a few more details. This flow chains the output of the first LLM block into the second and then from the second into the third. Forcing the LLM to process the problem this way uncovers the reasoning pattern, but it can also be overly verbose.

Listing 10.12 `prompt-chaining` flow

```
   Inputs:
        statement  : the statement problem to be solved

   LLM: decompose_steps (the prompt used to decompose the problem)
        inputs:
               statement: -> input.statement     #1

        outputs: the breakdown of steps to solve the problem

   LLM: calculate_steps (the prompt used to calculate the steps)
        inputs:
               statement: -> input.statement
               decompose_steps: -> output.decompose_steps     #2

               outputs: the calculation for each step
   LLM: calculate_solution (attempts to solve the problem)
        inputs:
               statement: -> input.statement
               decompose_steps: -> output.decompose_steps
               calculate_steps: -> output.calculate_steps     #3

         outputs: the final solution statement

   Outputs:
        statement: -> input.statement
        decompose_steps: -> output.decompose_steps
        calculate_steps: -> output.calculate_steps
        calculate_solution: -> output.calculate_solution

   ### Example Output
{
    "calculate_steps": "1. The days spent by Alex",
    "decompose_steps": "To figure out the …",
    "solution": "Alex spends 13 days in the ↪
           ↪ past before the end of the battle.",     #4
    "statement": "In a sci-fi film, Alex …"
}
```

#1 Start of the chain of prompts
#2 Output from the previous step injected into this step
#3 Output from two previous steps injected into this step
#4 The final solution statement, although wrong, is closer.

Run the flow by pressing Shift-F5 from the visual editor, and you'll see the output as shown in listing 10.12. The answer is still not correct for the Alex